HAN_UNIVERSITY OF APPLIED SCIENCES

PERSISTENCE WORKSHOP STEP-BY-STEP GUIDE[ONDERWERP]_

Game Minor – HAN University of Applied Sciences

Team Persistence

29 October 2026

CONTENTS

1 INTRODUCTION 2
1.1 What is persistence and what does it involve? 3
1.2 How does the workshop work (step-by-step plan) 3
1.3 What do you need? (Unity-version, Git) 3

2 PREPERATION 3
2.1 Cloning the repository 3
2.2 Opening the project in Unity 4
2.3 Starting the protype and seeing what doesn't work yet 5
2.4 Import and check third-party package (Quicksave) 5

3 WHAT IS INCLUDED AND WHAT IS MISSING? 5
3.1 The game 5
3.2 Overview of the layers that have been removed 5
3.3 Verification 5
3.4 What do you need to add? 6
3.4.1 Things you need for the implementation 6

4 REBUILDING THE PERSISTENCE LAYERS 7
4.1 Saving the username 7
4.2 Creating a save file 7
4.3 Loading a save file 7
4.4 Save summary and deleting save files 7
4.5 Handling damaged files 7
4.6 Extra: encrypting the save files 8

5 WRAP-UP: THE FINAL RESULT 8
5.1 Demo of complete prototype with all layers 9

1 INTRODUCTION

1.1 What is persistence and what does it involve?

Persistence means that data survives after closing the game. Without it, everything resets every time the game starts. Player position, progress, settings, profile information and choices would be lost upon closing.

Building persistence involves more than saving a file. In this workshop you will work with several layers:

- What to save: the data that matters, such as the player's position, progress, settings, and profile.
- How to store it: the format and location of the data, for example JSON file in Unity's Application.persistentDataPath.
- When to save and load: for example, when the player presses a button, switches save slots or closes and starts the game.
- How to organise it: profiles, save slots, and a menu that lets the user manage them.

By the end, you will understand how these layers fit together and be able to build them yourself.

1.2 How does the workshop work (step-by-step plan)

You will receive a working prototype from which the persistence layers have been removed. Your job is to rebuild them step by step.

1. Prepare: clone the project, open it, and see what already works.
2. Explore. find out which layers are missing and what you need to add.
3. Build: rebuild the persistence layers one at a time.
4. Finish: end with demo of the complete prototype.

1.3 What do you need? (Unity-version, Git)

- Unity 6000.5.10f1
- Git: clone the repository.
- A code editor with unity support.
- Internet of course to clone and import the third-party package.

2 PREPERATION

2.1 Cloning the repository

Open the terminal in the folder where you want to store the project and run:

git clone https://github.com/StijnWelles/persistence-workshop.git

Or open the repository directly using this link: https://github.com/StijnWelles/persistence-workshop

2.2 Opening the project in Unity

1. Open the Unity Hub.
2. Click Add and choose Add project from disk.
3. Select the folder you just cloned.
4. Open the project with Unity version 6000.5.10f1

2.3 Starting the protype and seeing what doesn't work yet

1. Open the scene "Main Scene" from the Project window.
2. Press Play.
3. Fill in a name, start a new game and walk around (WASD or arrow keys). Pick up some coins.
4. Press F5 (quicksave) and F9 (quickload), and press Esc to go back to the menu.
5. Quit and start the game again. Your name is gone, Continue is disabled and the slot list is empty. Nothing is saved yet.

2.4 Import and check third-party package (Quicksave)

The workshop uses QuickSave to write and read the save files. It is already included in the project under Assets/QuickSave, so you do not have to import anything.

- Wait until Unity has finished compiling.
- Check that the Console (Window > General > Console) shows no errors. Warnings about the missing saves are fine.

3 WHAT IS INCLUDED AND WHAT IS MISSING?

3.1 The game

The project has a small game where the player collects coins and must get a key to unlock the door to the gem. The game is playable and the UI is finished, but saving and loading is not functional.

3.2 Overview of the layers that have been removed

2 scripts need to be edited:

- Assets/Scripts/Persistence/SettingsStore.cs
- Assets/Scripts/Persistence/SaveSystem.cs

In these scripts, there are TODOs for every step.

3.3 Verification

There are unit tests per step to verify whether or not your solution is satisfactory.

In your IDE, you should be able to run the unit tests if you have the project open in Unity (Assets/Editor/PersistenceTests.cs > Run All).
If that doesn't work: In Unity, open Window > General > Test Runner > EditMode, press Run All.

Play the game from the Main Scene. The save files are saved as JSON, you can find the path by running the test PrintSaveFolder.

3.4 What do you need to add?

There is further explanation in the next chapter for each step.

Step	What you write	Test group	What you see in the game
1	SettingsStore.Save and Load	Step1_SettingsTests	Your name is still there after a restart
2	SaveSystem.Save and Exists	Step2_SaveTests	F5 creates a JSON file
3	SaveSystem.Load	Step3_LoadTests	F9 restores position, coins and door
4	ReadSummary and Delete	Step4_SummaryAndDeleteTests	The slot list and Continue work
5	try/catch in steps 1 to 4	Step5_DamagedFileTests	A damaged file does not crash the game
Extra	Encryption in SaveSystem	Step6_ExtraEncryptionTests	The JSON is unreadable

3.4.1 Things you need for the implementation

These are the QuickSave classes you need:

- QuickSaveWriter: creates or opens a file, writes values under a key and saves the file.
- QuickSaveReader: opens a file and reads the values back by key.
- QuickSaveException: thrown when something goes wrong, for example with a damaged file.
- Example code: Assets/QuickSave/Example/ExampleSceneManagerController.cs.

4 REBUILDING THE PERSISTENCE LAYERS

4.1 Saving the username

The goal that you want to achieve is for the game to show the name you entered in the menu, even after restarting the game.

The file you need to be working in is "SettingStore.cs". Also take a look at "Assets/QuickSave/Example/ExampleSceneManagerController.cs" to see how read and write functions work.

> **Tip:** Read throws an exception if the key is missing, TryRead returns false instead. Use TryRead here, so an older file without the name does not cause an error. RootExists is needed first, otherwise opening a missing file also throws an exception. Without Commit() nothing is written to the file.

Check: run Step1_SettingsTests. In the game, start a new game with your name, quit and open the New game screen again.

4.2 Creating a save file

The goal here is to create a JSON file by pressing F5 in the game.

The file you want to look at is "SaveSystem.cs", functions Save and Exists.

> **Tip:** The writer methods *can* be chained: Create(...).Write(...).Write(...).Commit(). The summary is saved separately so the slot list can show the name and coins without loading the whole game.

Check: run Step2_SaveTests. In the game, press F5 and look for the new file in the save folder.

4.3 Loading a save file

The goal here is to load the game by pressing F9. This puts the player, coins and door back as they were when they were saved.

The file you want to look at is "SaveSystem.cs", function Load.

Check: run Step3_LoadTests. In the game, press F5, move away, collect a coin and press F9.

4.4 Save summary and deleting save files

The goal is that the slot list shows the save and the delete button works.

The file you want to look at is "SaveSystem.cs".

Check: run Step4_SummaryAndDeleteTests. In the game, save to a few slots with Esc and open the Load screen. Restart the game and press Continue.

4.5 Handling damaged files

The goal is that a damaged save file does not crash the game.

The files you want to look at are "SettingsStore.cs" and "SaveSystem.cs"

> **Tip:** QuickSaveWriter.Create will open and attempt to parse the file if it exists. If the file format is broken, the writer will throw an exception even if your code is not explicitly reading the saved file.

Check: run Step5_DamagedFileTests. Then run the whole test suite. All tests of steps 1 to 5 should now pass.

4.6 Extra: encrypting the save files

The goal is to encrypt the save files to make tampering with the saves less likely.

The files you want to look at are "SaveSystem.cs".

> **Tip:** QuickSaveReader.Create can have a second parameter.
>
> Tip: Delete the current JSON files to avoid conflict between encrypted and non-encrypted files.

Check: run Step6_ExtraEncryptionTests. Open the JSON files and observe what happend to them.

5 WRAP-UP: THE FINAL RESULT

5.1 Demo of complete prototype with all layers

Play through the game once to check that all parts work together:

- Enter a name, start a game and restart: the name is still there.
- F5 and F9 restore your position, coins and door.
- Quitting to the menu creates an autosave and the Continue button loads it.
- The Load screen lists all slots with name, coins and time.
- Put some text in a save file to damage it: the game shows a message instead of crashing.

OPEN UP
NEW HAN_UNIVERSITY OF APPLIED SCIENCES
HORIZONS.